KnackELT Architecture

How knack-elt works, and where it sits in a full Knack → warehouse → BI stack.

Four diagrams:

1. Reference architecture — the end-to-end stack, in plain language and with the technical detail
2. Pipeline flow — what knack-elt actually does on a run
3. Run sequence — the call order for one invocation
4. SCD2 row lifecycle — how history is represented, and how to query it

> **Scope note.** This repo is the **ingestion** piece only. The dbt models, BI configuration, and CI orchestration shown in diagram 1 are drawn from a working production deployment where they live in a companion application repo (whose `pipelines/knack_dlt.py` is the production twin of this repo's pipeline). They are real, not aspirational — but they are not in this tree. Subgraph labels mark what lives where.

---

1. Reference architecture

Knack is the system of record and the operational UI. It is not a warehouse: no SQL, no aggregates, no history, and a record cap that eventually forces pruning. The stack below turns it into one — MotherDuck as the warehouse, dbt as the transformation layer, Preset (managed Apache Superset) as the access layer.

Same architecture, drawn twice: once for a stakeholder audience, once with the detail an engineer needs.

The stack in plain language


Where the work happens
Knack
the app the team uses every day
bookings · invoices · people · payments
read-only, nightly
Getting the data out
knack-elt
a nightly, automatic copy
keeps every version of every record
and never overwrites the old one
The data warehouse — MotherDuck
Complete history
every record Knack has ever had —
including records later deleted there
scheduled backup (optional)
OPTIONAL — offsite backup
S3-compatible object storage
AWS S3 · Cloudflare R2 · Backblaze B2 · MinIO
a dated copy kept outside the warehouse,
for disaster recovery only
Reporting tables — prepared by **dbt**
Cleaned up
dates, dollar amounts and linked records
put into a form you can add up and sort
Business views
payroll · events · people · invoices
shaped around the questions people ask
Where people get answers — Preset
Dashboards
look up a person, an event, a payment —
including ones no longer in Knack
Ad-hoc questions + export
new questions without a developer
results out to Excel or CSV

How to read it. The copy runs one way only — nothing this stack does can change data in Knack. The warehouse keeps history, so a record deleted in Knack is still answerable afterwards; that is the whole point of having it. The two boxes inside the warehouse are a division of labour, not two copies of the data: the complete history is what arrives, and **dbt** is the tool that turns it into the cleaned, business-shaped tables people actually query.

The dashed box is genuinely optional. The warehouse is already a second copy of Knack, so this is a third — insurance against losing the *warehouse* (a bad migration, an account problem, a provider outage), not against losing Knack. Add it when the warehouse becomes the only remaining copy of pruned records; skip it while Knack still holds everything. Any S3-compatible store works, and retention is normally handled by the bucket's own lifecycle rules rather than by code.

The same stack, with the technical detail


Source of record
Knack App
low-code UI + database
Knack REST API
api.knack.com/v1
daily cron — GitHub Actions
(companion repo)
1. dlt sync, one retry on failure
2. dbt build + data tests
3. offsite Parquet export (optional)
paginated REST
1000 rows/page
triggers
Ingestion — knack-elt (THIS REPO)
dlt pipeline
extract · normalize · load
merge, strategy = scd2
primary key = record_id
Warehouse — MotherDuck (DuckDB cloud)
raw schema — dlt writes here
knack_{stable_app_id}_data.{stable_app_id}
object_N tables · field_N columns
full SCD2 history
_load_info · _trace
_kn_object_catalog · _kn_field_catalog
observability + current labels
dbt run
knack-elt refresh-views
explicit, asks first
scheduled Parquet export
(optional)
reporting schema — the tables people query
defined and built by dbt (companion repo)
staging views (stg_*)
one per source table
SCD2 flags · currency + date casts
connection-array extraction
mart views
consumer-facing, documented
live / deleted posture
_labels schema — views only
{stable_app_id}_labels
current-label names over raw · two views per object
disposable — refresh-views rebuilds, run-pipeline only reports drift
OPTIONAL — disaster recovery
S3-compatible object storage
AWS S3 · Cloudflare R2 · Backblaze B2 · MinIO
dated Parquet snapshots, lifecycle-managed
SQL over MotherDuck
Access layer
Preset
managed Apache Superset
SQL Lab
ad-hoc + query development
Dashboards
lookup · drill-down · analytics
CSV / Excel export

The load-bearing detail: schema ownership is split.

Schema	Owner	Contents	Rule
raw (the stable app-id dataset)	dlt	One `object_N` table per Knack object, `field_N` columns, full SCD2 history	**dbt never builds into it.** It is the pipeline's output, and a `dbt run` writing here would be clobbered on the next sync.
`{stable_app_id}_labels`	knack-elt (`labels.py`)	Two views per object — `"Customers"` (live rows) and `"Customers_history"` (every version, plus `valid_from` / `valid_to` / `is_live_in_knack`)	Views only, disposable, rebuilt wholesale by `knack-elt refresh-views` — never automatically, never touching `object_N` / `field_N`.
`reporting`	dbt	Staging + mart views	Everything downstream reads here. No dashboard queries raw directly.

Label views are this repo's, not the companion repo's. `{stable_app_id}_labels` is generated from `_kn_object_catalog` / `_kn_field_catalog` — the labels loaded on every sync — so an analyst can browse `"Customers"` instead of `object_3`. It is disposable and read-only: identifier comparisons are ASCII-case folded so `"Customers"` and `"customers"` can't collide, but every name emitted into SQL is verbatim, and an apply is a full drop-and-rebuild in one transaction rather than an incremental diff, because an incremental create-then-drop can transiently clobber a view if two objects swap the names their views hold. Nothing rebuilds this layer on its own — a label edited in the Knack builder must never move a warehouse name by itself. `knack-elt refresh-views` prints a plan and asks before applying it; `run-pipeline` only reports drift at the end of a sync and changes nothing. A hand-authored view in the schema is removed on the next apply; a hand-authored table blocks the rebuild instead and is named in the error.

Staging exists so that exactly one layer absorbs the mess Knack and dlt produce together — currency strings with commas, dates wrapped in JSON, connection fields as JSON arrays, and dlt's type-variant columns (when a value doesn't fit the type inferred from the first batch, dlt NULLs the base column and diverts the value to a sibling like `amount__v_double`, which makes a naive `SUM()` under-report silently). Marts and dashboards then get to be simple.

Why Preset: it is managed Superset, so SQL Lab doubles as the query-development environment and the dashboard builder against the same MotherDuck connection — a proven query becomes a dataset becomes a chart with no rewrite. Any BI tool with a DuckDB/MotherDuck SQLAlchemy connection substitutes here; the contract is the `reporting` schema, not the tool. Note that row-level security and embedded analytics are paid-tier features in Preset — untrusted multi-tenant self-service needs a different surface (a scoped API in front of MotherDuck), not a BI seat.

---

2. Pipeline flow

What one run of knack-elt does. Two inputs from Knack: the **app metadata** (schema) and the **records** (data). Immutable keys identify physical tables and columns; metadata labels are loaded into catalog tables for discovery.

FIGURE 3 — PIPELINE FLOW

Notes on the mapping pass

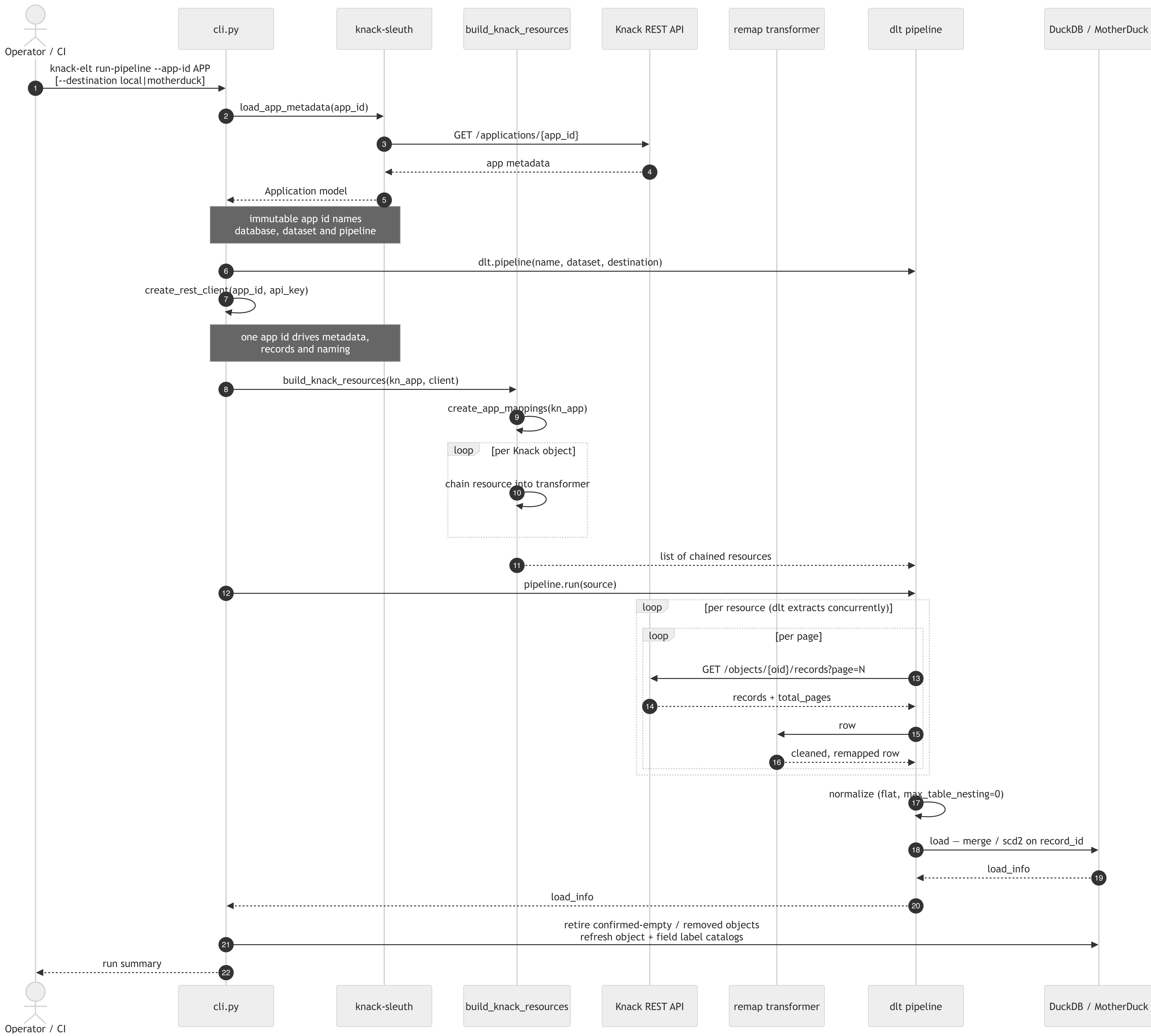
- Object and field labels are editable metadata, so they never identify physical warehouse objects. `object_12` remains table `object_12`; `field_73` remains column `field_73`.
- `_kn_object_catalog` and `_kn_field_catalog` map those stable keys to current labels for discovery and downstream model generation.
- Label changes, duplicate/non-Latin labels, and names matching top-level system keys therefore cannot split history or overwrite a value.
- Knack's row id arrives as the top-level `id` key and is renamed to `record_id`, which is the pipeline's primary key. A field labelled "ID" remains under its immutable `field_N` column. The key is taken from the payload, not from Knack's auto-added "Record ID" field, so it works identically on apps that predate that field.
- Cleaning runs **before** the remap only, so it matches on raw Knack field keys. `numeric_fields` and `default_values` are registered under those raw keys only.
- `remap_keys` falls back to the original key when no mapping exists, so an unmapped object still loads — with `field_NN` column names.

---

3. Run sequence

The packaged entry point is the Typer CLI (`knack-elt run-pipeline`). It takes the app metadata from knack-sleuth rather than fetching it itself, and builds the record client from the *same* app id and key, so metadata and records always describe one app.

FIGURE 4 — RUN SEQUENCE



One entry point, two destinations:

`--destination`	Target	Requires
`local` (default)	a DuckDB file at `$XDG_DATA_HOME/knack-elt/knack_{stable_app_id}_data.duckdb` (falling back to `~/.local/share/knack-elt/`), or wherever `--db-path` points. Deliberately not working-directory relative — one app, one warehouse, wherever the command runs	nothing — this is the zero-setup path, so a fresh install can load a Knack app straight away
`motherduck`	`md:///knack_{stable_app_id}_data`	`motherduck_api_key` in the environment or `.env`

Either way the run sets `load.workers = 3` and `truncate_staging_dataset = True`, reconciles confirmed-empty and removed objects, and writes `_load_info`, `_trace`, `_kn_object_catalog` and `_kn_field_catalog` beside the data. Last, if a `{stable_app_id}_labels` schema already exists, it reports any label drift since those views were last built — read-only, and never able to change the run's exit code. See the label views section above for what builds and rebuilds that schema.

The application id and REST API key are threaded from the CLI into the record client, so `--app-id` alone fully determines which app is read — metadata and records cannot disagree.

---

4. SCD2 row lifecycle

Every table is loaded with `write_disposition={"disposition": "merge", "strategy": "scd2"}`, so the warehouse is **append-and-retire, not overwrite**. dlt adds `_dlt_valid_from` and `_dlt_valid_to` to every row. A record's history is therefore several rows sharing one `record_id`.

FIGURE 5 — SCD2 ROW LIFECYCLE

Because of that last case, two different flags are needed, and conflating them is the most common way to get a wrong answer out of this stack:

```
-- one row per record, live or deleted, plus the live/deleted flag
with flagged as (
    select
        *,
        row_number() over (partition by record_id order by _dlt_valid_from desc) = 1
            as latest_version,
        _dlt_valid_to is null as is_live_in_knack
    from acme_ops.some_table         -- the raw, dlt-owned schema
)
select * from flagged where latest_version
```

Question	Filter
Current state of everything still in Knack	`latest_version and is_live_in_knack`
Latest known state of every record ever seen, including deleted	`latest_version`
What did this record look like on a given date	`_dlt_valid_from <= d and (_dlt_valid_to is null or _dlt_valid_to > d)`
What has been deleted from Knack	`latest_version and not is_live_in_knack`

Two traps worth stating explicitly:

- **Partition by `record_id`, not by a column that looks like an id.** `record_id` is the canonical Knack row id, taken from the API payload and populated on every row. A `field_N` column may hold the app's own numbering or Knack's auto-added Record ID field, but neither is the merge key. Partitioning on either collapses rows into wrong groups and corrupts the flags.
- **Aggregate without `latest_version` and you double-count.** Every historical version of a row is still a row. A `SUM()` over the raw table sums every version of every record.

Deleted-in-Knack is not the same as *archived by policy* — distinguishing an intentional retention prune from incidental deletion needs a separate manifest of what the pruning process removed; SCD2 alone cannot tell them apart.

Configuration

`pydantic-settings`, loaded from environment or `.env` (`src/knack_elt/config.py`):

Variable	Used for
`KNACK_APP_ID`	Knack application id — also the default for `--app-id`
`KNACK_API_KEY`	Knack REST API key (sent as `X-Knack-REST-API-Key`)
`motherduck_api_key`	MotherDuck token, interpolated into the `md:///` connection string

Note that dbt-duckdb reads the MotherDuck token from `motherduck_token`, not `motherduck_api_key` — the two layers disagree on the variable name, so a deployment running both needs to set or shim both.